



Politecnico di Torino

# GPU Programming 01URVOV

Master's Degree in Computer Engineering

## Shortest path algorithm for GPU parallelization Project Report

Candidate:

Fabio Delbosco (S322244)

Referee:

Prof. Riccardo Cantoro

Dr. Josie Esteban Rodriguez Condia

Dr. Juan David Guerrero Balaguera

---

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction Chapter</b>                                       | <b>1</b>  |
| <b>2</b> | <b>Graph Representation and Shortest Path Algorithms</b>          | <b>2</b>  |
| 2.0.1    | Graph Representations . . . . .                                   | 2         |
| 2.0.2    | Sequential Dijkstra Algorithm . . . . .                           | 3         |
| 2.0.3    | Delta-Stepping Algorithm . . . . .                                | 3         |
| <b>3</b> | <b>Experimental Platform</b>                                      | <b>5</b>  |
| <b>4</b> | <b>Implementation Analysis</b>                                    | <b>6</b>  |
| 4.1      | Baseline Implementation: Dijkstra with Adjacency Matrix . . . . . | 6         |
| 4.2      | Optimization with Shared Memory and Reduction . . . . .           | 7         |
| 4.3      | Kernel Fusion to Reduce Launch Overhead . . . . .                 | 7         |
| 4.4      | Limitations of the Adjacency Matrix Approach . . . . .            | 7         |
| 4.5      | Switching to CSR and Delta-Stepping . . . . .                     | 7         |
| 4.6      | Delta-Stepping parallelization . . . . .                          | 8         |
| 4.6.1    | CUDA Implementation . . . . .                                     | 8         |
| 4.6.2    | Atomic Operations and Memory Access . . . . .                     | 8         |
| 4.6.3    | Shared Memory Aggregation . . . . .                               | 8         |
| 4.6.4    | Choice of $\Delta$ . . . . .                                      | 9         |
| 4.6.5    | Profiling summary . . . . .                                       | 9         |
| <b>5</b> | <b>Results analysis</b>                                           | <b>11</b> |
| <b>6</b> | <b>Scalability analysis</b>                                       | <b>14</b> |
| <b>A</b> | <b>Conclusions</b>                                                | <b>15</b> |

---

# List of Figures

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 2.1 | Dense adjacency matrix and Compressed Sparse Row (CSR) comparison . . . . . | 2  |
| 2.2 | Delta stepping Vertex edge bucketization . . . . .                          | 3  |
| 5.1 | Dijkstra execution time Vs total vertices graph . . . . .                   | 12 |
| 5.2 | Dijkstra average execution time . . . . .                                   | 12 |
| 5.3 | Delta-Stepping Vertex edge bucketization . . . . .                          | 12 |
| 5.4 | Delta-Stepping Vertex edge bucketization histogram . . . . .                | 12 |
| 5.5 | Delta stepping Vertex edge bucketization . . . . .                          | 13 |

---

---

## CHAPTER 1

---

# Introduction Chapter

The computation of shortest paths in weighted graphs is a fundamental problem in computer science, with applications ranging from communication networks to large-scale data analysis. Classical algorithms, such as Dijkstra’s algorithm, compute the shortest paths from a source node to all other nodes. However, their traditional CPU implementations suffer from bottlenecks caused by the serial selection of the minimum-distance node at each iteration, and the use of dense structures such as adjacency matrices, which do not scale well for large graphs.

With the advent of GPUs, it is possible to parallelize the algorithm across hundreds of cores, drastically reducing execution times. Nevertheless, GPU efficiency heavily depends on data layout, memory coalescing, and minimizing warp divergence.

In this work, two main GPU-based approaches for shortest path computation were implemented: a parallelized version of Dijkstra optimized with shared memory and block-wise reductions, and a Delta-Stepping version for sparse graphs, which leverages buckets to process light and heavy edges concurrently. The study compares the performance of both solutions on randomly generated graphs of various sizes, highlighting their advantages, limitations, and potential optimizations.

---

## CHAPTER 2

---

# Graph Representation and Shortest Path Algorithms

### 2.0.1 Graph Representations

In this work, I considered two standard representations of graphs:

- **Adjacency Matrix:** A dense  $V \times V$  matrix  $A$  where  $A[i][j]$  stores the weight of the edge  $(i, j)$  if it exists, or 0 otherwise. This representation allows constant-time access to any edge, but requires  $O(V^2)$  memory, which becomes prohibitive for large graphs.
- **Compressed Sparse Row (CSR):** CSR is a space-efficient representation suited for sparse graphs. It uses three arrays:
  - $row\_ptr[V + 1]$  indicates the start of each vertex's adjacency list in the other arrays.
  - $col\_ind[E]$  stores the destination vertices of edges.
  - $weights[E]$  stores the corresponding edge weights.

Memory usage is  $O(V + E)$ , which is much more scalable for sparse graphs.

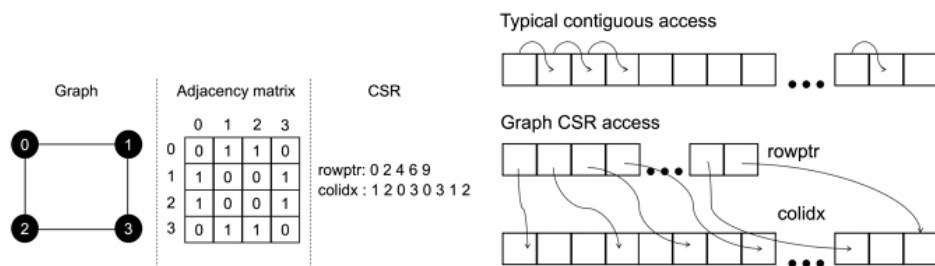


Figure 2.1: Dense adjacency matrix and Compressed Sparse Row (CSR) comparison

In my study, I initially implemented Dijkstra using an adjacency matrix for simplicity, but later switched to CSR to handle larger graphs efficiently.

### 2.0.2 Sequential Dijkstra Algorithm

Dijkstra's algorithm computes the shortest path from a source vertex  $s$  to all other vertices in a weighted graph with non-negative edges. The algorithm maintains a tentative distance array  $dist[v]$  and a set of visited nodes  $sptSet[v]$ .

1. **Initialization:**

$$dist[s] = 0, \quad dist[v] = \infty \text{ for all } v \neq s$$

2. **Minimum Selection:** At each step, select the unvisited node  $u$  with minimum  $dist[u]$ .

3. **Edge Relaxation:** For each neighbor  $v$  of  $u$ :

$$dist[v] = \min(dist[v], dist[u] + w(u, v))$$

4. **Mark  $u$  as visited:**  $sptSet[u] = \text{true}$

**Complexity:**

- Using an adjacency matrix:  $O(V^2)$  time,  $O(V^2)$  memory.
- Using CSR with a priority queue:  $O((V + E) \log V)$  time,  $O(V + E)$  memory.

The main limitation of Dijkstra is the need for global minimum selection, which serializes the algorithm and creates a bottleneck for parallelization.

### 2.0.3 Delta-Stepping Algorithm

Delta-Stepping is an alternative shortest path algorithm that processes vertices in *distance buckets*. Edges are classified into light and heavy based on a parameter  $\Delta$ :

$$\text{Light edges: } w(u, v) \leq \Delta, \quad \text{Heavy edges: } w(u, v) > \Delta$$

1. **Bucket Assignment:** Each node  $v$  is placed in a bucket according to its tentative distance:

$$bucket[v] = \left\lfloor \frac{tent[v]}{\Delta} \right\rfloor$$

2. **Light Phase:** Nodes in the current bucket relax all outgoing light edges in parallel. Nodes whose distance changes may remain in the same bucket or move to a future bucket.

3. **Heavy Phase:** After all light edges are relaxed, heavy edges of the processed nodes are relaxed.

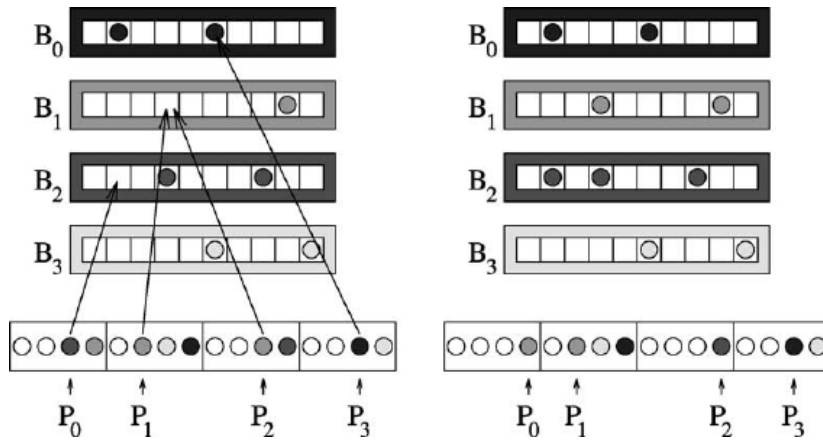


Figure 2.2: Delta stepping Vertex edge bucketization

**Complexity:**

- Memory:  $O(V + E)$ , similar to CSR.
- Time: Each edge is relaxed at most once. Let  $E_L$  and  $E_H$  denote the number of light and heavy edges respectively. Then the theoretical cost is:

$$O(E_L + E_H)$$

- Best suited for sparse graphs; reduces the dependency on global minimum selection.

**Parallelization potential:**

Delta-Stepping lends itself well to parallel execution because:

- Relaxation of light edges in a bucket can be done concurrently by multiple threads.
- Shared memory can be used to store per-block frontier nodes, reducing atomic operations in global memory.
- Heavy edges are handled separately, allowing load balancing and avoiding serialization.

---

## CHAPTER 3

---

# Experimental Platform

All CUDA implementations presented in this work were developed and tested using **CUDA Toolkit 11.8**. The hardware target for experiments was a **NVIDIA GTX 750** GPU, based on the **Maxwell architecture**. The key hardware characteristics relevant to my study are:

- **Global Memory:** 2 GB of internal GPU memory.
- **Shared Memory:** 48 KB per Streaming Multiprocessor (SM).
- **Streaming Multiprocessors:** 4 SMs.
- **Maximum Thread Concurrency:** Only one kernel can be executed concurrently per SM.

These specifications impose constraints on both the size of the graphs that can be processed and the way memory is allocated and accessed within kernels. In particular:

- The limited global memory restricts the use of adjacency matrices for large graphs.
- The shared memory per SM is sufficient to implement per-block buffers for delta-stepping frontiers, but careful management is required to avoid overflow.
- The single concurrent kernel execution per SM can limit occupancy and parallel throughput for certain kernels, making efficient memory access patterns crucial.

All experiments and performance profiling were carried out on this setup, ensuring that the reported results reflect realistic constraints for mid-range GPU architectures.

The CPU used to run the code was an **Intel Core i7-6700 CPU @ 3.41 GHz**, equipped with **16 GB of RAM**. This CPU handled the sequential parts of the algorithms, data preparation, and memory transfers between host and device, providing a baseline for CPU vs GPU performance comparison.



---

---

## CHAPTER 4

---

# Implementation Analysis

### 4.1 Baseline Implementation: Dijkstra with Adjacency Matrix

The initial implementation focused on the classical Dijkstra algorithm using an adjacency matrix representation. The adjacency matrix was stored as a two-dimensional array of `short` integers in order to reduce memory consumption, while still allowing sufficiently large edge weights. The algorithm follows the traditional structure:

1. Initialize all distances as  $\infty$ , except for the source node, set to 0.
2. At each iteration, select the vertex  $u$  with minimum tentative distance that has not yet been processed.
3. For each neighbor  $v$ , update the tentative distance according to the relaxation step:

$$\text{dist}[v] \leftarrow \min\{\text{dist}[v], \text{dist}[u] + w(u, v)\}$$

where  $w(u, v)$  is the weight of the edge  $(u, v)$ .

4. Repeat until all vertices have been processed.

The time complexity of this naive approach with adjacency matrices is:

$$O(V^2)$$

both in terms of memory and computation, since each iteration requires scanning all  $V$  vertices to identify the minimum distance node.

The initial expectation was that a direct CUDA parallelization would lead to significant speedups. However, profiling revealed the algorithm to be heavily **memory-bound**, as the ratio between operations and memory accesses was extremely low. For instance, considering the cost of distance updates, the computational work was estimated as:

$$W = 5 + 3V^2, \quad Q = 11 + 12V^2 - 1$$

with an arithmetic intensity of:

$$\frac{W}{Q} \approx 0.35$$

which indicates that the algorithm spends more time accessing memory than performing arithmetic computations. Consequently, the naive CUDA implementation did not yield meaningful speedups, as the overhead of memory transfers dominated execution.

## 4.2 Optimization with Shared Memory and Reduction

The most computationally expensive step of Dijkstra is the search for the minimum tentative distance among all unvisited nodes. In the adjacency matrix version, this step required accessing all  $V$  entries at each iteration, leading to poor scaling.

To mitigate this, a kernel was designed that loads chunks of the distance array into **shared memory**, enabling each block to perform a local reduction to identify the minimum value. The per-block minima were then combined into a global minimum. The data was split into two parts in shared memory to avoid exceeding per-block memory limits.

This approach significantly reduced global memory accesses and improved coalescing, achieving up to a **20% speedup** on large graphs with more than 25,000 nodes.

## 4.3 Kernel Fusion to Reduce Launch Overhead

Another critical issue was the sequential nature of Dijkstra. Each iteration requires:

1. Finding the minimum distance node.
2. Relaxing its neighbors.
3. Marking the node as visited.

A naive CUDA implementation involved launching up to three kernels per iteration, leading to high overhead. To alleviate this, the kernel responsible for finding the local minimum was fused with the global minimum calculation kernel, reducing the number of kernel launches per iteration. This modification yielded further performance improvements, with speedups of **30–40%** for graphs with 30,000 vertices.

## 4.4 Limitations of the Adjacency Matrix Approach

Despite these optimizations, fundamental limitations remained:

- The memory requirement grows as  $O(V^2)$ , making adjacency matrices infeasible for very large graphs.
- Even with short integers, storing weights for large graphs quickly exhausted GPU global memory.
- The algorithm remained constrained by its inherently sequential nature, preventing efficient parallel scaling.

## 4.5 Switching to CSR and Delta-Stepping

Given these constraints, the adjacency matrix representation and Dijkstra's algorithm were deemed unsuitable for large-scale graphs. Instead, the work shifted to:

- A **Compressed Sparse Row (CSR)** representation of the graph, which stores only existing edges, reducing memory from  $O(V^2)$  to  $O(V + E)$ .
- The **Delta-Stepping** algorithm, which avoids strict sequential dependencies by processing nodes in buckets, making it more amenable to parallelization.

This change of direction was crucial for enabling experiments on graphs with tens of thousands of vertices and hundreds of edges per vertex, where Dijkstra was no longer practical.

## 4.6 Delta-Stepping parallelization

The *Delta-Stepping* algorithm is a parallel shortest path algorithm that replaces the strict ordering of Dijkstra’s priority queue with a relaxed, bucket-based structure. Tentative distances are grouped into buckets of width  $\Delta$ , and buckets are processed sequentially, but all vertices within a bucket can be relaxed concurrently. This relaxation of the total order enables coarse-grain parallelism that maps well to GPUs.

### 4.6.1 CUDA Implementation

The GPU implementation consists of three main kernels:

1. **kernel\_light\_step**: processes all light edges from the current frontier. Each thread handles one vertex  $u$ , iterating over its adjacency list and attempting to update tentative distances via

$$tent[v] \leftarrow \min(tent[v], tent[u] + w(u, v)).$$

Updates require `atomicMin` to avoid races.

2. **kernel\_heavy\_step**: processes heavy edges from the set of vertices  $S$  discovered in the current bucket. These updates are less frequent but also require atomic operations.
3. **kernel\_collect\_bucket**: gathers vertices whose tentative distance places them in the next bucket, compacting them into the next frontier.

The light-step kernel is the most critical: it performs the majority of relaxations and atomic updates, and its performance depends heavily on the distribution of edge weights and vertex degrees.

### 4.6.2 Atomic Operations and Memory Access

Atomic operations are essential for correctness since multiple threads may attempt to relax the same destination vertex  $v$ . However, they introduce two performance issues:

- **Contention**: multiple threads writing to the same memory location serialize, reducing throughput.
- **Non-coalesced accesses**: destination indices  $v$  are irregular, causing scattered global memory writes and reducing memory efficiency.

These effects explain the relatively low warp execution efficiency and global memory efficiency observed during profiling.

### 4.6.3 Shared Memory Aggregation

To mitigate contention, I introduced a block-level aggregation scheme:

- Each block accumulates candidate vertices for the next frontier in a shared-memory buffer (*s\_buf*).
- Threads use shared atomics (`atomicAdd`, `atomicMin`) to update local counters and minima.
- At the end of the kernel, a single thread per block performs the corresponding global atomics to update the global frontier and next bucket index.

This significantly reduces the number of global atomics but introduces some overhead:

- Copying the shared buffer into global memory (“flush”) adds synchronization and memory traffic.
- If the shared buffer overflows, threads must fall back to direct global atomics.

As a result, the shared-memory version improves performance in cases with moderate contention but can be slightly slower when the flush overhead dominates.

#### 4.6.4 Choice of $\Delta$

The value of  $\Delta$  controls the trade-off between parallelism and the number of iterations.

- A very small  $\Delta$  produces many buckets (close to Bellman-Ford behaviour), increasing synchronization overhead.
- A very large  $\Delta$  approaches Dijkstra’s algorithm, reducing parallelism within each bucket.

In practice, I use the heuristic:

$$\Delta \approx \frac{\mathbb{E}[w]}{2},$$

where  $\mathbb{E}[w]$  is the average edge weight. This choice tends to balance the number of buckets with the expected intra-bucket parallelism: most typical weights fall into the same bucket, ensuring that many relaxations can be performed concurrently, while still keeping the number of buckets manageable.

#### 4.6.5 Profiling summary

In summary, Delta-Stepping is well suited for GPUs because it scales with the number of edges rather than vertices, and it naturally exposes parallelism through bucket-based relaxation. The main challenges are atomic contention and irregular memory access, which I partially mitigate through shared-memory buffering. The choice of  $\Delta$  is critical to balance the number of iterations and the amount of parallelism, and my heuristic based on half the average edge weight provided a good balance in practice.

| Kernel / Metric           | Dijkstra    | Delta-Stepping<br>1k | Delta-Stepping<br>20k | Delta-Stepping<br>Shared 20k |
|---------------------------|-------------|----------------------|-----------------------|------------------------------|
| Achieved Occupancy        | 0.39–0.48   | 0.018–0.50           | 0.019–0.98            | 0.017–0.995                  |
| Warp Execution Efficiency | 92–98%      | 8–71%                | 5–71%                 | 5–71%                        |
| Global Load Efficiency    | 64–96%      | 12–13%               | 12–12.7%              | 12–13.3%                     |
| Global Store Efficiency   | 0–97%       | 0–99%                | 0–42%                 | 0–99%                        |
| Global Load Throughput    | 44–209 MB/s | 4.9 MB/s–895 MB/s    | 43 MB/s–7.38 GB/s     | 4.5 MB/s–9.21 GB/s           |
| Global Store Throughput   | 0–65 MB/s   | 0–5 MB/s             | 0–66 MB/s             | 0–24 MB/s                    |
| Instructions per Warp     | 35–129      | 634–30k              | 116–12k               | 106–11k                      |
| Issue Slot Utilization    | 16–23%      | 0.5–3.6%             | 0.52–3.2%             | 0.83–25%                     |
| Executed IPC              | 0.73–1.10   | 0.068                | 0.025–0.161           | 0.039–1.09                   |

Table 4.1: Comparative CUDA metrics for Dijkstra and Delta-Stepping implementations on small (1k nodes) and large (20k nodes) graphs. Metrics include achieved occupancy, warp execution efficiency, global memory efficiency, throughput, instructions per warp, issue slot utilization, and executed IPC. Shared memory versions of Delta-Stepping show improved store efficiency and reduced atomic contention.

Dijkstra exhibits stable occupancy and high warp efficiency (92–98%), reflecting regular memory access patterns. Delta-Stepping shows greater variability, especially in `kernel_light_step`, but occupancy improves on larger graphs due to increased parallelism.

Global load efficiency is low ( $\sim 12 - 13\%$ ) for Delta-Stepping light-step kernels, while shared memory aggregation boosts store efficiency on large graphs, sometimes near 99%. Throughput scales with graph size, reaching several GB/s for Delta-Stepping, whereas Dijkstra remains limited by smaller workloads.

Instruction-level metrics confirm that light-step kernels are irregular and memory-bound, with low IPC despite many instructions per warp. Shared memory mitigates contention and improves throughput in critical kernels.

Overall, these observations highlight that Dijkstra is predictable but memory-bound, while Delta-Stepping better exploits GPU parallelism on large graphs.

---

## CHAPTER 5

---

# Results analysis

### Analysis of Dijkstra and Delta-Stepping performance.

The tables above report the execution times and speedups used to generate the graphs. Three graph sizes were selected to exploit the maximum computational capability of Dijkstra while respecting the GPU's global memory constraint. The largest graph that could fit in the GPU memory consists of 30,000 vertices, which nearly reaches the 2 GB limit of global memory ( $30,000 \text{ vertices} \times 30,000 \text{ edges} \times 4 \text{ bytes} = 1.6 \text{ GB}$ ).

All graphs were generated using the 'CSRGraph' class method 'CSR\_generate()' and then converted to dense matrix by void 'CRS\_to\_dense(short\* graph, CSRGraph G)'. The number of vertices ('VERTICES'), the average degree per vertex ('AVG\_DEG'), and the maximum edge weight ('MAX\_WEIGHT = 100') can be modified in the 'cuda\_kernel.cuh' file to generate graphs with different sizes, densities, and edge weights. The same graphs were used to obtain the execution times for all algorithms, ensuring a fair and consistent comparison across implementations.

| Vertices | Edges per Vertex | CPU (ms)           | GPU (ms)                  |
|----------|------------------|--------------------|---------------------------|
| 10000    | 100              | 1026, 1045         | 1161, 1168                |
| 20000    | 200              | 1065, 1082         | 2825, 2774                |
| 30000    | 300              | 10780, 10968, 9673 | 4494.44, 4354.68, 4677.45 |

Table 5.1: Execution times (ms) for Dijkstra algorithm.

| Vertices | Edges per Vertex | CPU (ms)         | GPU (ms)            | GPU Shared (ms)     |
|----------|------------------|------------------|---------------------|---------------------|
| 10000    | 100              | 2472, 2425, 2411 | 19.44, 21.23, 21.23 | 22.67, 24.88, 24.88 |
| 20000    | 200              | 24645, 2435      | 20.80, 16.59        | 21.36, 25.49        |
| 30000    | 300              | 33730, 33706     | 105.99, 103.05      | 156.80, 165.94      |

Table 5.2: Execution times (ms) for Delta-Stepping algorithm.

**Dijkstra: CPU vs GPU.** The CPU execution times for Dijkstra scale relatively moderately from 10,000 to 20,000 vertices, showing a slight linear increase. However, when reaching 30,000 vertices, the CPU time jumps dramatically to over 10,000 ms, which indicates that Dijkstra on CPU suffers from scalability issues on larger graphs. On the other hand, the GPU implementation starts slightly slower for 10,000 vertices, likely due to kernel launch overheads and data transfer costs, but quickly

overtakes the CPU at 20,000 vertices. By 30,000 vertices, the GPU executes in less than half the time of the CPU, confirming that parallelization provides substantial benefits for larger graphs.

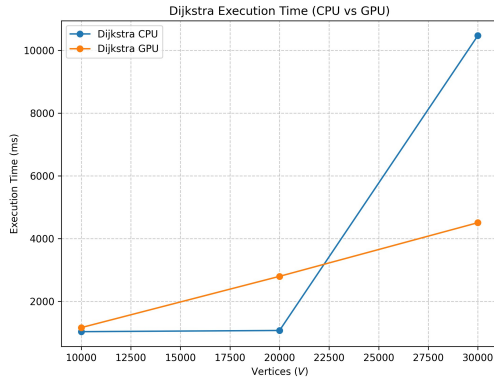


Figure 5.1: Dijkstra execution time Vs total vertices graph

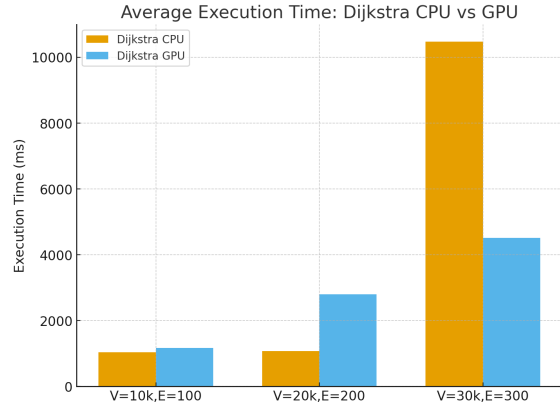


Figure 5.2: Dijkstra average execution time

**Delta-Stepping: CPU vs GPU.** Delta-Stepping on the CPU exhibits a sharp increase in execution time as vertex count rises, reflecting its computational intensity. The GPU version, in contrast, executes drastically faster, even for 20,000 vertices, where the CPU requires over 13,000 ms. The GPU shared memory optimization (*Delta GPU Shared*) shows a slightly higher time for smaller graphs due to overhead but scales more efficiently for larger graphs. This suggests that using shared memory becomes increasingly advantageous as graph size increases.

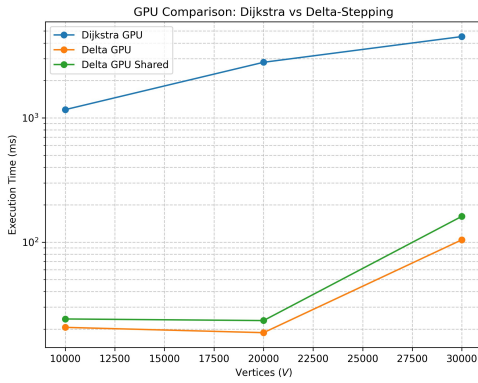


Figure 5.3: Delta-Stepping Vertex edge bucketization

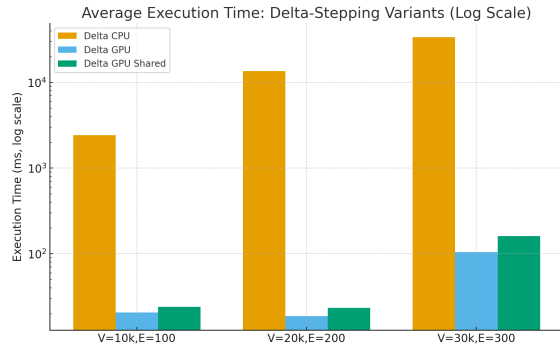


Figure 5.4: Delta-Stepping Vertex edge bucketization histogram

**GPU Comparison: Dijkstra vs Delta-Stepping.** When comparing GPU implementations, Delta-Stepping clearly outperforms Dijkstra across all tested sizes. For 20,000 vertices, Dijkstra on GPU takes almost 2,800 ms, whereas Delta-Stepping completes in under 20 ms—a difference of more than two orders of magnitude. Shared memory optimization further improves performance at the largest graph size, confirming that Delta-Stepping not only benefits from GPU parallelism but also from efficient memory management.

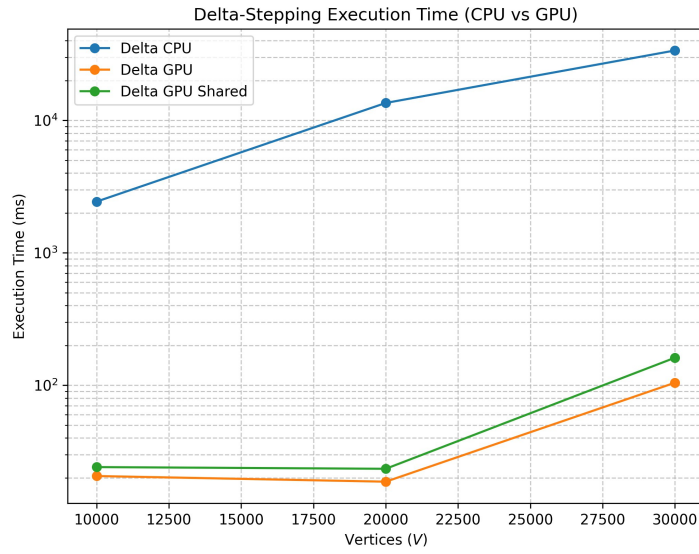


Figure 5.5: Delta stepping Vertex edge bucketization

**Summary Insight.** Overall, these results indicate that while Dijkstra may suffice for smaller graphs, Delta-Stepping combined with GPU acceleration provides superior performance for medium to large-scale graphs. The use of shared memory offers additional benefits for very large graphs, highlighting the importance of both algorithm choice and low-level GPU optimization.



---

---

## CHAPTER 6

---

# Scalability analysis

Scalability is a crucial aspect of shortest-path algorithms, particularly on GPU architectures with limited memory resources. In my experiments, I evaluate scalability considering both the number of edges and the memory footprint of the algorithms.

### Dijkstra.

- Uses an adjacency-matrix representation, consuming  $O(V^2)$  memory.
- Memory requirements rapidly become limiting for large graphs, even with shared memory optimizations.
- Execution time grows quadratically with the number of edges, since each iteration scans all vertices for the minimum distance.
- Overall, Dijkstra exhibits poor scaling both in memory and computation for large graphs.

### Delta-Stepping.

- Uses a CSR representation, storing only existing edges, resulting in  $O(V + E)$  memory usage.
- Scales efficiently to tens of thousands of vertices with hundreds of edges per vertex without exhausting GPU memory.
- Bucket-based parallel relaxation enables many threads to work concurrently, making execution time scale more gracefully with graph density.
- For large graphs, Delta-Stepping becomes compute-bound rather than memory-bound, unlike Dijkstra.

### Memory-Constrained Scenario.

- Considering a hypothetical scenario where only half of the GPU memory is usable, Dijkstra becomes even more limited due to its quadratic memory footprint.
- Delta-Stepping remains feasible under restricted memory, highlighting its suitability for large-scale graphs.

**Summary.** While Dijkstra quickly hits memory and performance bottlenecks as graph size increases, Delta-Stepping demonstrates robust scalability in both computation and memory usage, making it the preferred approach for GPU-based large-scale graph processing.

---

---

## APPENDIX A

---

# Conclusions

This work allowed me to explore and optimize shortest-path computation on graphs, investigating multiple algorithmic and data-structure approaches. In particular, Delta-Stepping emerged as the most suitable solution due to its scalability, both in terms of the underlying graph representation (CSR) and the algorithm itself, which naturally lends itself to parallel execution, as discussed in the previous sections and confirmed by the test results.

Extensive experiments on large graphs—up to 300,000 nodes with 300 edges per node—demonstrate how Delta-Stepping fully exploits GPU parallelism, reducing execution time dramatically: for example, from approximately 50 minutes on CPU to 2,667 ms on GPU.

Although the shared-memory implementation already improves performance, further optimization is possible. In particular, memory coalescing could be enhanced by sorting the frontier by destination node. While this introduces initial overhead on smaller graphs, it would likely provide significant benefits for very large graphs with millions of nodes and billions of edges.

One limitation of Delta-Stepping is non-uniform load balancing in highly skewed graphs, where nodes have widely varying degrees. A potential improvement would be to assign threads to edges instead of nodes, which could mitigate thread idling and improve throughput.

The hardware available allowed careful exploration of memory usage and data-structure optimization. However, for graphs that exceed device memory, it is challenging to implement memory tiling or graph partitioning effectively. Traversing and merging very large, highly connected subgraphs would introduce considerable overhead and complexity.

In summary, this project highlights the potential of GPU-accelerated Delta-Stepping for large-scale shortest-path computation. The algorithm’s inherent parallelism, combined with an optimized CSR representation, enables high scalability and significant speedups over CPU implementations, while leaving room for further improvements in memory access patterns and load balancing.